

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 3
з курсу “Операційні системи реального часу”
на тему: “Обмін даними між завданнями за допомогою черг у
FreeRTOS”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Мета роботи. Навчитися використовувати черги для обміну даними між завданнями.

```
// Налаштування пінів
```

```
/// Параметри черг та періодів (легко змінювати для експериментів)
```

```
SENSOR_PERIOD_MS 200 // початковий період зчитування сенсора  
##define SENSOR_PERIOD_MS 50 // для експерименту — спробувати 50 ms
```

```
// Черги
```

```
// Функція для відправки у чергу з перевіркою та логуванням при переповненні
```

```
// Якщо не змогли додати — повідомляємо у UART
```

```
// --- Завдання: кнопка з debounce ---
```

```
const TickType_t pollDelay = pdMS_TO_TICKS(20); // частота опитування для дебаунсу  
const uint32_t stableCountNeeded = 3; // 3 * 20ms = 60ms стабільності
```

```
bool lastState = true; // бо використовується pull-up: released = 1, pressed = 0
```

```
// стан не змінився — нарощуємо стабільний лічильник
```

```
// стан змінився — скидаємо лічильник
```

```
// реальний стабільний стан підтверджено
// реагуємо лише на спуск (pressed) якщо використовуємо pull-up

// надсилаємо без блокування (щоб задачі не чіплялися)

// зачекаємо поки користувач відпустить (щоб уникнути багаторазової відправки
поки тримає)
// але робити це простою затримкою: чекаємо поки кнопка стане released

// після відпускання даємо невеликий інтервал

// --- Завдання: обробник натискань LED ---

// Перемкнути стан простого індикатора (якщо бажаєте)

// --- Завдання: зчитування ADC (потенціометр) ---

// читаємо ADC (0..4095 для 12-bit як приклад; на RP2040 adc_read() дає 12-bit)
// Використовуємо adc_read() з Pico SDK (припускаємо попередню ініціалізацію

value = (uint16_t) raw; // зберігаємо як 16-bit
// відправляємо без блокування (щоб не зупиняти сенсор при переповненні)
```

```
// Якщо не вдалося — виводимо повідомлення
```

```
// --- Завдання: контроль яскравості через PWM ---
```

```
// PWM налаштовано в main
```

```
// wrap встановлений у main (65535)
```

```
    // Масштабуємо: ADC (0..65535) -> PWM level (0..wrap)  
duty = (uint32_t)sensorValue; // тут діапазон вже 0..65535
```

```
// Ініціалізація пінів
```

```
// для простого включення/вимкнення через gpio_put у vTaskLedEventHandler  
// але для PWM використовуємо pwm_gpio  
// Налаштуємо PWM для LED_GPIO:
```

```
// Ініціалізація ADC
```

```
// Створюємо черги
```

```
// Створення задач
xTaskCreate(vTaskButton, "Button", 256, NULL, 3, NULL);    // трохи вищий пріоритет
```

```
// Старт планувальника
```

```
// Якщо scheduler не стартував — зациклюємось
```

Як часто надходять нові значення?

Нові значення надходять з періодом, заданим у `vTaskSensor` — у коді це `SENSOR_PERIOD_MS` (за замовчуванням 200 ms). Отже, частота — $1 / (\text{SENSOR_PERIOD_MS}) = 5 \text{ Hz}$ при 200 ms.

Що відбувається, якщо `vTaskDelay()` у сенсорному завданні зменшити до 50 мс?

Зменшення періоду до 50 ms підвищує частоту генерації повідомлень до 20 Hz. Якщо споживач (`vTaskLedControl`) не встигає обробляти повідомлення (черга мала або обробка займає час), черга почне заповнюватися і за її межами `xQueueSend` поверне помилку — ми виводитимемо попередження про переповнення, повідомлення будуть втрачені (`send` не блокувався у прикладі). Якщо змінити відправку на блокуючу з `timeout`, сенсорна задача може блокуватися і падати її продуктивність.

Чи встигає споживач обробляти всі повідомлення?

Залежить від: розміру черги, швидкості надходження та часу обробки споживача. При початкових налаштуваннях (200 ms період, `queue len` 10) споживач з великою ймовірністю встигає. При 50 ms і маленькій черзі (наприклад 1) — ні, частина повідомлень буде втрачена або з'являться повідомлення про переповнення.

Як змінюється поведінка системи при різних розмірах черги (1, 5, 20)?

- `queue len = 1` — майже нульовий буфер: якщо споживач тимчасово зайнятий, пропадуть повідомлення, часто побачите `WARN/ERROR` про

переповнення (залежно від логіки send). Система більш «чутлива» до затримок споживача.

- `queue len = 5` — невеликий буфер: згладжує короткі пікові навантаження, але при частому надходженні (50 ms) може переповнитись.
- `queue len = 20` — великий буфер: дозволяє акумулювати багато повідомлень, зменшує втрати при короткочасних «сплесках», але займає більше RAM в системі (кожен елемент пам'ять). Також при великому буфері може з'явитися «затримка» між реальним моментом вимірювання і моментом обробки (старі значення обробляються пізніше).

Повідомлення у UART при переповненні черги

В коді є дві точки, де ми виводимо повідомлення:

- `safeQueueSend()` — якщо кнопкова черга переповниться.
- В `vTaskSensor` — якщо `xQueueSend(sensorQueue, &value, 0)` повертає не `pdTRUE`, ми виводимо `"WARN: sensorQueue overflowed (value %u lost)\n"`.

Висновок: у ході лабораторної роботи було реалізовано обмін даними між задачами FreeRTOS за допомогою черг, а також досліджено їхню поведінку при різних параметрах. Під час виконання завдань було створено систему з кнопкою, сенсорним зчитуванням і керуванням яскравістю світлодіода, що дозволило на практиці оцінити роботу черг. Експерименти з періодом надсилання даних та розміром черги показали, як змінюється продуктивність системи та виникають переповнення. Отримані результати підтвердили важливість правильного вибору частоти опитування та глибини черг для стабільної роботи системи реального часу.